

MODULE-3

Day-1

Beginner:

Exercise-1:

Create a menu driven program that:

- Accepts a user input for n students (name, roll no, grade).
- Stores them in a structured format (array / list of objects).
- Displays data in tabular format using formatted output.

```
from tabulate import tabulate

# List to store student data
students = []

# Function to add students
def add_students():
    n = int(input("Enter number of students to add: "))
    for i in range(n):
        print(f"\nEnter details for Student {i+1}")
        name = input("Name : ")
        roll_no = input("Roll No : ")
        grade = input("Grade : ")
        students.append({
            "Name": name,
            "Roll No": roll_no,
            "Grade": grade
        })
    print(f"\n{n} student(s) added successfully.\n")

# Function to display all students
def display_students():
    if not students:
```

```

        print("\n No student data to display.\n")
    else:
        print("\n Student Records:\n")
        print(tabulate(students, headers="keys", tablefmt="grid"))
# Menu-driven loop
def main():
    while True:
        print("\n===== Student Management Menu =====")
        print("1. Add Student(s)")
        print("2. Display All Students")
        print("3. Exit")
        choice = input("Enter your choice (1-3): ")
        if choice == '1':
            add_students()
        elif choice == '2':
            display_students()
        elif choice == '3':
            print("Exiting the program. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter 1, 2, or 3.")

```

Exercise-2:

Write a program with global variable, local variable and a static variable inside a function. Trace their values across multiple function calls.

```

# Global variable
counter = 0

def demo_function():
    global counter # Accessing global variable

```

```

counter += 1
local_var = 10 # Local variable
# Simulating a static variable using function attribute
if not hasattr(demo_function, "static_var"):
    demo_function.static_var = 100 # Initialize static variable
demo_function.static_var += 5 # Modify static variable
print(f'Function Call {counter}:')
print(f' Global Variable (counter): {counter}')
print(f' Local Variable (local_var): {local_var}')
print(f' Static Variable (static_var): {demo_function.static_var}')
print("-" * 40)
# Call the function multiple times
for i in range(3):
    demo_function()

```

Intermediate:

Exercise-1:

Write a program that demonstrates your understanding of arrays, functions, and parameter passing by completing the following steps using C++, Java, or Python:

- Define a function that takes an array (or list) of integers and its size (if applicable) as parameters.
- The function should iterate through the array to determine and return the maximum element.
- In the main program, prompt the user to input the size of the array and its elements.
- Pass the array to the defined function and display the maximum value returned by the function.

```

def find_max(arr):
    if not arr:
        return None # handle empty list
    max_val = arr[0]
    for num in arr:
        if num > max_val:
            max_val = num
    return max_val

# Main program
def main():
    size = int(input("Enter the size of the array: "))
    if size <= 0:
        print("Invalid size. Array must have at least one element.")
        return
    arr = []
    print(f"Enter {size} integer elements:")
    for i in range(size):
        num = int(input(f"Element {i+1}: "))
        arr.append(num)
    # Call the function to find maximum
    max_value = find_max(arr)
    # Display result
    print(f"\nThe maximum value in the array is: {max_value}")

# Run the program
main()

```

Exercise 2:

Demonstrate your understanding of recursion by solving classic problems that require a recursive approach. Implement the following tasks using appropriate programming constructs in C++, Java, or Python.

a) Find the nth Fibonacci number using recursion

- Accept an integer n from the user.
- Use a recursive function to compute

b) Solve the Tower of Hanoi problem for n disks

- Accept the number of disks n from the user.
- Write a recursive function that prints the sequence of moves to transfer all disks from the source rod to the destination rod, using an auxiliary rod.

a) # Recursive function to find nth Fibonacci number

```
def fibonacci(n):
```

```
    if n <= 0:
```

```
        return 0
```

```
    elif n == 1:
```

```
        return 1
```

```
    else:
```

```
        return fibonacci(n-1) + fibonacci(n-2)
```

```
# Main for Fibonacci
```

```
def main_fibonacci():
```

```
    n = int(input("Enter the value of n to find nth Fibonacci number: "))
```

```
    result = fibonacci(n)
```

```
    print(f"The {n}th Fibonacci number is: {result}")
```

b) # Recursive function for Tower of Hanoi

```
def tower_of_hanoi(n, source, destination, auxiliary):  
    if n == 1:  
        print(f"Move disk 1 from {source} to {destination}")  
        return  
    tower_of_hanoi(n - 1, source, auxiliary, destination)  
    print(f"Move disk {n} from {source} to {destination}")  
    tower_of_hanoi(n - 1, auxiliary, destination, source)  
  
# Main for Tower of Hanoi  
def main_hanoi():  
    n = int(input("\nEnter number of disks for Tower of Hanoi: "))  
    print("\nSequence of moves:")  
    tower_of_hanoi(n, 'A', 'C', 'B') # Source: A, Destination: C, Auxiliary: B
```

Advanced:

Exercise 1:

Design a student record system using both procedural and object-oriented programming paradigms, and provide a comparative analysis of the two approaches.

- **Procedural Approach** : Develop the system using a procedural programming approach. Use global data structures to store student records and write functions to perform operations such as adding, updating, deleting, and displaying student details.
- **Object-Oriented Approach** : Redesign the same student record system using object-oriented programming principles. Create classes such as Student and RecordManager to encapsulate data and behavior. Implement the same operations using methods within these classes.
- **Comparison** : Critically compare the procedural and object-oriented implementations of your student record management

system. Evaluate them in terms of code reusability, maintainability, scalability, readability, and ease of implementation.

Procedural Approach:

```
# Global list to store student records

students = []

# Function to add student
def add_student():
    name = input("Enter name: ")
    roll = input("Enter roll number: ")
    grade = input("Enter grade: ")
    students.append({'name': name, 'roll': roll, 'grade': grade})
    print("Student added successfully.\n")

# Function to update student
def update_student():
    roll = input("Enter roll number of student to update: ")
    for student in students:
        if student['roll'] == roll:
            student['name'] = input("Enter new name: ")
            student['grade'] = input("Enter new grade: ")
            print("Student updated successfully.\n")
            return
    print("Student not found.\n")

# Function to delete student
def delete_student():
    roll = input("Enter roll number to delete: ")
    global students
```

```
students = [s for s in students if s['roll'] != roll]

print("Student deleted (if existed).\n")

# Function to display all students
def display_students():
    if not students:
        print("No records found.\n")
        return
    print("\nStudent Records:")
    for s in students:
        print(f"Name: {s['name']}, Roll: {s['roll']}, Grade: {s['grade']}")
    print()

# Menu
def procedural_menu():
    while True:
        print("---- Procedural Student Record System ----")
        print("1. Add Student")
        print("2. Update Student")
        print("3. Delete Student")
        print("4. Display All")
        print("5. Exit")
        choice = input("Enter choice: ")
        if choice == '1':
            add_student()
        elif choice == '2':
            update_student()
        elif choice == '3':
            delete_student()
```

```

elif choice == '4':
    display_students()
elif choice == '5':
    break
else:
    print("Invalid choice\n")
# Run procedural version
# procedural_menu()

```

Object-Oriented Approach :

class Student:

```

def __init__(self, name, roll, grade):
    self.name = name
    self.roll = roll
    self.grade = grade
def __str__(self):
    return f"Name: {self.name}, Roll: {self.roll}, Grade: {self.grade}"

```

class RecordManager:

```

def __init__(self):
    self.students = []
def add_student(self):
    name = input("Enter name: ")
    roll = input("Enter roll number: ")
    grade = input("Enter grade: ")
    self.students.append(Student(name, roll, grade))
    print("Student added successfully.\n")

```

```

def update_student(self):
    roll = input("Enter roll number to update: ")
    for s in self.students:
        if s.roll == roll:
            s.name = input("Enter new name: ")
            s.grade = input("Enter new grade: ")
            print("Student updated successfully.\n")
            return
    print("Student not found.\n")
def delete_student(self):
    roll = input("Enter roll number to delete: ")
    self.students = [s for s in self.students if s.roll != roll]
    print("Student deleted (if existed).\n")
def display_students(self):
    if not self.students:
        print("No student records.\n")
        return
    print("\nStudent Records:")
    for s in self.students:
        print(s)
    print()
# Menu
def oop_menu():
    manager = RecordManager()
    while True:
        print("---- OOP Student Record System ----")
        print("1. Add Student")

```

```

print("2. Update Student")
print("3. Delete Student")
print("4. Display All")
print("5. Exit")
choice = input("Enter choice: ")
if choice == '1':
    manager.add_student()
elif choice == '2':
    manager.update_student()
elif choice == '3':
    manager.delete_student()
elif choice == '4':
    manager.display_students()
elif choice == '5':
    break
else:
    print("Invalid choice\n")
# Run OOP version
# oop_menu()

```

Practice Assignment-1:

Task-1:

```
class Employee:
```

```
    def __init__(self, employee_id: int, name: str, position: str, salary: float,
performance_rating: float):
```

```
        self.employee_id = employee_id
```

```
        self.name = name
```

```
        self.position = position
```

```

        self.salary = float(salary)

        self.performance_rating = float(performance_rating)

    def update_details(self, name=None, position=None, salary=None,
rating=None):
        if name: self.name = name
        if position: self.position = position
        if salary is not None: self.salary = float(salary)
        if rating is not None: self.performance_rating = float(rating)

    def display_details(self):
        print(self)

    def __str__(self):
        return (f'Employee ID: {self.employee_id}, Name: {self.name}, Position:
{self.position}, "

                f'Salary: ₹{self.salary:.2f}, Performance Rating:
{self.performance_rating:.1f}')

```

Task-2:

```

class PerformanceReview:

    def __init__(self, employee_id: int, review_date: str, rating: float, comments:
str):
        self.employee_id = employee_id
        self.review_date = review_date # Format: YYYY-MM-DD
        self.rating = float(rating)
        self.comments = comments

    def update_review(self, rating=None, comments=None):
        if rating is not None:
            self.rating = float(rating)
        if comments:
            self.comments = comments

```

```
def display_review(self):  
    print(self)  
def __str__(self):  
    return (f"Employee ID: {self.employee_id}, Date: {self.review_date}, "  
           f"Rating: {self.rating:.1f}, Comments: {self.comments}")
```

Task-3: Not Completed.

Day-2

Beginner:

Exercise-1:

Build a learning platform with multiple course types:

- OnlineCourse (self-paced/live sessions)
- OfflineCourse (classroom schedule)
- HybridCourse (combination of both)

Tasks:

- Create a base class Course with a method getSchedule().
- Override getSchedule() in each course type to print their specific schedule.
- Store different course objects in a list or array of base class references.
- Loop through the list and call getSchedule() using runtime polymorphism.

class Course:

```
def getSchedule(self):  
    print("General course schedule")
```

```

class OnlineCourse(Course):
    def __init__(self, mode):
        self.mode = mode # 'self-paced' or 'live'
    def getSchedule(self):
        if self.mode.lower() == 'self-paced':
            print("Online Course: Access anytime (self-paced).")
        elif self.mode.lower() == 'live':
            print("Online Course: Attend live sessions as per schedule.")
        else:
            print("Online Course: Invalid mode specified.")
class OfflineCourse(Course):
    def getSchedule(self):
        print("Offline Course: Attend classes at campus as per fixed schedule.")
class HybridCourse(Course):
    def getSchedule(self):
        print("Hybrid Course: Attend online and offline sessions as per
combination schedule.")
# Create course objects
course_list = [
    OnlineCourse("self-paced"),
    OnlineCourse("live"),
    OfflineCourse(),
    HybridCourse()
]
# Runtime polymorphism
for course in course_list:
    course.getSchedule()

```

Exercise 2:

Define an abstract base class Appliance and create a hybrid structure to implement Smart Fridge which inherits from both Cooling Device and Wi-Fi Enabled.

```
from abc import ABC, abstractmethod

# Abstract base class
class Appliance(ABC):
    @abstractmethod
    def operate(self):
        pass

# First base class: Cooling capabilities
class CoolingDevice:
    def cool(self):
        print("Cooling system activated.")

# Second base class: IoT capabilities
class WiFiEnabled:
    def connect_wifi(self):
        print("Connected to Wi-Fi network.")

# Hybrid inheritance: SmartFridge
class SmartFridge(Appliance, CoolingDevice, WiFiEnabled):
    def operate(self):
        print("Smart Fridge is now operating.")
        self.cool()
        self.connect_wifi()

# Driver code
fridge = SmartFridge()
fridge.operate()
```

Intermediate:

Exercise 1:

You're helping a small retail store automate its billing system. Different types of products have different billing methods, but all products share some common characteristics, like a name and a price. You need to design a system that can calculate the final bill for different types of products, such as regular items, discounted items, and tax-exempt items, using object-oriented inheritance principles. Build a system with the following structure:

1. Base class Product

- Attributes: name, price
- Method: `get_final_price()` – returns the base price

2. Subclasses:

- Regular Product: No discount or tax. Uses base price.
- Discounted Product: Applies a fixed discount (e.g., 10% off).
- Tax Exempt Product: Applies no tax.
- Taxed Product: Adds a fixed tax rate (e.g., 18%).

3. Each subclass must override `get_final_price()` to reflect its own pricing logic.

```
# Base class
```

```
class Product:
```

```
    def __init__(self, name, price):
```

```
        self.name = name
```

```
        self.price = price
```

```
    def get_final_price(self):
```

```
        return self.price
```

```
# Subclass 1: Regular Product
```

```
class RegularProduct(Product):
```

```

    def get_final_price(self):
        return self.price
# Subclass 2: Discounted Product (10% off)
class DiscountedProduct(Product):
    def get_final_price(self):
        return self.price * 0.90 # 10% discount
# Subclass 3: Tax Exempt Product (same as base)
class TaxExemptProduct(Product):
    def get_final_price(self):
        return self.price
# Subclass 4: Taxed Product (18% tax)
class TaxedProduct(Product):
    def get_final_price(self):
        return self.price * 1.18 # 18% tax added
# Testing different products
product_list = [
    RegularProduct("Notebook", 100),
    DiscountedProduct("Pen", 50),
    TaxExemptProduct("Children's Book", 200),
    TaxedProduct("Electronics", 1000)
]
# Billing system loop
for product in product_list:
    print(f'{product.name}: ₹{product.get_final_price():.2f}')

```

Exercise 2:

You are designing a personnel system for a modern organization that supports on-site and remote employees. Employees have general human attributes (like name), and depending on their role and location, they may also have specialized attributes. Model an employee system:

- Human (base class with name)
 - Employee (extends Human, adds employee_id)
 - Remote Worker (has location)
 - Remote Employee inherits from both and prints all details
- Implement this system using object-oriented principles, particularly focusing on inheritance and constructor chaining in the context of hybrid inheritance.

```
class Human:
```

```
    def __init__(self, name):  
        self.name = name  
        print(f"[Human] Name: {self.name}")
```

```
class Employee(Human):
```

```
    def __init__(self, name, employee_id):  
        super().__init__(name)  
        self.employee_id = employee_id  
        print(f"[Employee] ID: {self.employee_id}")
```

```
class RemoteWorker:
```

```
    def __init__(self, location):  
        self.location = location  
        print(f"[RemoteWorker] Location: {self.location}")
```

```
class RemoteEmployee(Employee, RemoteWorker):
```

```
    def __init__(self, name, employee_id, location):
```

```

# Constructor chaining for multiple inheritance

Employee.__init__(self, name, employee_id) # calls Human via super()

RemoteWorker.__init__(self, location)

def display_details(self):

    print("\n--- Remote Employee Details ---")

    print(f"Name: {self.name}")

    print(f"Employee ID: {self.employee_id}")

    print(f"Location: {self.location}")

```

Advanced:

Exercise 1:

You are part of a robotics company tasked with building a control system for different types of robots. Every robot is expected to support some default behaviours, such as `move()` and `stop()`. However, depending on its type or use case, a robot may also possess specialized capabilities such as flying or swimming, or both.

- The system must be designed with flexibility and reusability in mind.
- Robot behaviour should be able to change dynamically at runtime based on new functionality or mission updates.
- The architecture must use object-oriented principles, particularly inheritance and method overriding, or optionally, strategy-based behaviour modelling.
- Add logic to detect and handle unsupported actions.

```

from abc import ABC, abstractmethod

```

```

class FlyBehavior(ABC):

```

```

    @abstractmethod

```

```

    def fly(self):

```

```

        pass
class SwimBehavior(ABC):
    @abstractmethod
    def swim(self):
        pass
# Strategy Implementations
class CanFly(FlyBehavior):
    def fly(self):
        print("Flying through the sky.")
class CannotFly(FlyBehavior):
    def fly(self):
        print("Error: This robot cannot fly.")
class CanSwim(SwimBehavior):
    def swim(self):
        print("Swimming in water.")
class CannotSwim(SwimBehavior):
    def swim(self):
        print("Error: This robot cannot swim.")
#Base Robot Class
class Robot:
    def __init__(self, name):
        self.name = name
        self.fly_behavior = CannotFly()
        self.swim_behavior = CannotSwim()
    def move(self):
        print(f"{self.name} is moving forward.")

```

```

def stop(self):
    print(f"{self.name} has stopped.")
def fly(self):
    print(f"{self.name} attempt to fly:")
    self.fly_behavior.fly()
def swim(self):
    print(f"{self.name} attempt to swim:")
    self.swim_behavior.swim()
# Dynamic behavior change
def set_fly_behavior(self, fly_behavior):
    self.fly_behavior = fly_behavior
def set_swim_behavior(self, swim_behavior):
    self.swim_behavior = swim_behavior

```

Practice Assignment:

Task-1:

```

class Train:
    def __init__(self, train_id, train_name, capacity):
        self.train_id = train_id
        self.train_name = train_name
        self.capacity = capacity
    def update_details(self, train_name=None, capacity=None):
        if train_name:
            self.train_name = train_name
        if capacity:
            self.capacity = capacity

```

```
def display(self):
    print(f"Train ID: {self.train_id}")
    print(f"Train Name: {self.train_name}")
    print(f"Capacity: {self.capacity}")

# Example Usage
train1 = Train(101, "Express", 300)
train1.display()
print("Updating train details...\n")
train1.update_details(train_name="Super Express", capacity=350)
train1.display()
```

Task-2:

```
class Train:
    def __init__(self, train_id, train_name, capacity):
        self.train_id = train_id
        self.train_name = train_name
        self.capacity = capacity
    def update_train(self, train_name=None, capacity=None):
        if train_name:
            self.train_name = train_name
        if capacity:
            self.capacity = capacity
    def display(self):
        print(f"Train ID: {self.train_id}")
        print(f"Train Name: {self.train_name}")
        print(f"Capacity: {self.capacity}")
class PassengerTrain(Train):
```

```

def __init__(self, train_id, train_name, capacity, number_of_coaches):
    super().__init__(train_id, train_name, capacity)
    self.number_of_coaches = number_of_coaches

def update_passenger_train(self, train_name=None, capacity=None,
number_of_coaches=None):
    self.update_train(train_name, capacity)
    if number_of_coaches:
        self.number_of_coaches = number_of_coaches

def display(self):
    super().display()
    print(f"Number of Coaches: {self.number_of_coaches}")

# Example Usage
pt = PassengerTrain(101, "Chennai Express", 1200, 16)
pt.display()
print("\nAfter update:\n")
pt.update_passenger_train(capacity=1300, number_of_coaches=18)
pt.display()

```

Task-3:

Base Class

class Train:

```

def __init__(self, train_id, train_name, capacity):
    self.train_id = train_id
    self.train_name = train_name
    self.capacity = capacity

def update_details(self, train_name=None, capacity=None):
    if train_name:
        self.train_name = train_name

```

```

        if capacity:
            self.capacity = capacity
    def display_details(self):
        print(f"Train ID: {self.train_id}")
        print(f"Train Name: {self.train_name}")
        print(f"Capacity: {self.capacity}")
# Derived Class - Freight Train
class FreightTrain(Train):
    def __init__(self, train_id, train_name, capacity, cargo_capacity):
        super().__init__(train_id, train_name, capacity)
        self.cargo_capacity = cargo_capacity
    def update_freight_details(self, train_name=None, capacity=None,
cargo_capacity=None):
        self.update_details(train_name, capacity)
        if cargo_capacity is not None:
            self.cargo_capacity = cargo_capacity
    def display_freight_details(self):
        self.display_details()
        print(f"Cargo Capacity: {self.cargo_capacity} tons")
# Sample usage
freight = FreightTrain(202, "Goods Express", 50, 120.5)
freight.display_freight_details()
freight.update_freight_details(cargo_capacity=150.75)
freight.display_freight_details()

```

Task-4:Not Completed.

Day-3

Beginner:

Exercise-1:

You are working on a software module for a scientific instrument control system. Some operations, like data reading, may come in different forms (e.g., reading as integers, floats, or from external sources). At the same time, different types of instruments may behave differently when it comes to processing or displaying that data. Your task is to design a class (or set of classes) that demonstrates both:

- Function Overloading (Compile-Time Polymorphism)
- Virtual Function Overriding (Runtime Polymorphism)

```
from abc import ABC, abstractmethod

# Base class
class Instrument(ABC):

    # Function Overloading using default arguments and type checking
    def read_data(self, data=None):
        if isinstance(data, int):
            print(f'Reading integer data: {data}')
        elif isinstance(data, float):
            print(f'Reading float data: {data}')
        elif isinstance(data, str):
            print(f'Reading data from source: {data}')
        else:
            print("No data provided or unknown type")

    # Virtual function (overridden by child classes)
    @abstractmethod
    def process_data(self):
```

```

        pass
# Derived class 1
class Thermometer(Instrument):
    def process_data(self):
        print("Processing temperature data in Thermometer.")
# Derived class 2
class Spectrometer(Instrument):
    def process_data(self):
        print("Processing light spectrum data in Spectrometer.")
# Test the implementation
if __name__ == "__main__":
    # Function overloading simulation
    base = Thermometer()
    base.read_data(100)
    base.read_data(98.6)
    base.read_data("SensorPort-A")
    # Virtual function overriding
    instruments = [Thermometer(), Spectrometer()]
    for instr in instruments:
        instr.process_data()

```

Exercise 2:

Build foundational understanding of polymorphism in object-oriented programming by simulating behavior through a shared interface.

Tasks:

- Create a base class or interface `Animal` with a method `makeSound()`.

- Implement derived classes like Dog, Cat, and Cow, each overriding makeSound() to print a different sound.
- Demonstrate polymorphism by calling makeSound() using a common reference or interface.

```
from abc import ABC, abstractmethod

# Base class / Interface
class Animal(ABC):
    @abstractmethod
    def makeSound(self):
        pass

# Derived class: Dog
class Dog(Animal):
    def makeSound(self):
        print("Dog says: Woof!")

# Derived class: Cat
class Cat(Animal):
    def makeSound(self):
        print("Cat says: Meow!")

# Derived class: Cow
class Cow(Animal):
    def makeSound(self):
        print("Cow says: Moo!")

# Polymorphic demonstration
def animal_sound_demo():
    # List of Animal references pointing to different derived class objects
    animals = [Dog(), Cat(), Cow()]
    for animal in animals:
```

```

        animal.makeSound() # Polymorphic call

# Run the demo
if __name__ == "__main__":
    animal_sound_demo()

```

Intermediate:

Exercise-1:

You are tasked with designing a Math utility system for a scientific computing application. The system should support basic arithmetic operations (like add, subtract) and advanced mathematical operations (like integrate, differentiate).

The objective is to demonstrate how both compile-time and runtime polymorphism can coexist in a single design to build a scalable and flexible utility system.

```

from abc import ABC, abstractmethod
from functools import singledispatchmethod
import sympy as sp

# Base Class: MathUtility (Abstract)
class MathUtility(ABC):
    @singledispatchmethod
    def add(self, a, b):
        raise NotImplementedError("Unsupported types")
    @add.register
    def _(self, a: int, b: int):
        return a + b

```

```

    @add.register
    def _(self, a: float, b: float):
        return a + b

    def subtract(self, a, b):
        return a - b

    @abstractmethod
    def integrate(self, expression, variable):
        pass

    @abstractmethod
    def differentiate(self, expression, variable):
        pass

# Derived Class: SymbolicMath
class SymbolicMath(MathUtility):
    def integrate(self, expression, variable):
        var = sp.Symbol(variable)
        expr = sp.sympify(expression)
        return sp.integrate(expr, var)

    def differentiate(self, expression, variable):
        var = sp.Symbol(variable)
        expr = sp.sympify(expression)
        return sp.diff(expr, var)

# Derived Class: NumericMath
class NumericMath(MathUtility):
    def integrate(self, expression, variable):
        raise NotImplementedError("Numeric integration not supported in this
class.")

    def differentiate(self, expression, variable):

```

```
        raise NotImplementedError("Numeric differentiation not supported in this
class.")
```

```
# Demo
```

```
def demo():
```

```
    sm = SymbolicMath()
```

```
    nm = NumericMath()
```

```
    # Compile-time polymorphism: Overloaded add()
```

```
    print("Add integers:", sm.add(5, 3))
```

```
    print("Add floats :", sm.add(2.5, 3.7))
```

```
    # Runtime polymorphism: Overridden methods
```

```
    expr = "x**2 + 3*x + 2"
```

```
    print("Symbolic Integrate:", sm.integrate(expr, 'x'))
```

```
    print("Symbolic Differentiate:", sm.differentiate(expr, 'x'))
```

```
    print("Numeric Subtract:", nm.subtract(10, 4))
```

```
    # Attempting runtime override in base-only feature
```

```
    try:
```

```
        nm.integrate(expr, 'x')
```

```
    except NotImplementedError as e:
```

```
        print("Handled Error:", e)
```

```
# Run the demo
```

```
if __name__ == "__main__":
```

```
    demo()
```

Exercise-2:

Develop a modular image processing application that supports applying various filters to an image. These filters include effects such as:

- Blur

- Sharpen
- Grayscale

The system should be designed to be extensible, so that new filters (like Sepia, Invert Colors, etc.) can be added in the future without modifying existing code. This calls for the use of polymorphism and a plugin-friendly architecture.

Requirements:

- Define a Base Class or Interface (ImageFilter)
- Implement Derived Classes like BlurFilter, SharpenFilter, and Grayscale Filter.
- Each class should override the process() method to provide specific Behaviour for the filter.
- Use Polymorphism for Dynamic Filter Application
- Emphasize that the core processing logic doesn't need to change when new filters are introduced.

```
from abc import ABC, abstractmethod
from PIL import Image, ImageFilter, ImageOps

# Base Interface: ImageFilterBase
class ImageFilterBase(ABC):
    @abstractmethod
    def process(self, image):
        pass

# Derived Filters
class BlurFilter(ImageFilterBase):
    def process(self, image):
        return image.filter(ImageFilter.BLUR)

class SharpenFilter(ImageFilterBase):
    def process(self, image):
```

```

        return image.filter(ImageFilter.SHARPEN)

class GrayscaleFilter(ImageFilterBase):
    def process(self, image):
        return image.convert("L")

# Example: Adding Sepia Filter (Extensible)
class SepiaFilter(ImageFilterBase):
    def process(self, image):
        grayscale = image.convert("L")
        sepia = ImageOps.colorize(grayscale, "#704214", "#C0A080")
        return sepia

# Example: Invert Filter
class InvertFilter(ImageFilterBase):
    def process(self, image):
        return ImageOps.invert(image)

# Filter Application Engine
class ImageProcessor:
    def __init__(self, filter_obj: ImageFilterBase):
        self.filter = filter_obj

    def apply_filter(self, image_path, output_path):
        img = Image.open(image_path)
        processed_img = self.filter.process(img)
        processed_img.save(output_path)
        print(f"Saved: {output_path}")

# Demo
def demo():
    image_path = "sample.jpg"

```

```

output_dir = "output_"
# Filters
filters = {
    "blur": BlurFilter(),
    "sharpen": SharpenFilter(),
    "grayscale": GrayscaleFilter(),
    "sepia": SepiaFilter(),
    "invert": InvertFilter()
}
for name, filter_obj in filters.items():
    processor = ImageProcessor(filter_obj)
    processor.apply_filter(image_path, f"{output_dir}{name}.jpg")
if __name__ == "__main__":
    demo()

```

Advanced:

Exercise-1:

Create a messaging platform that needs to support multiple types of notifications to users. These include:

- Email Notifications
- SMS Notifications
- Push Notifications Each of these types has a different implementation for how a message is sent. Your task is to design a scalable and maintainable system using polymorphism so that the notification mechanism is easily extendable in the future.

Requirements:

1. Create a base class or interface called Notification with a method send(String message).

2. Derive concrete classes (Email Notification, SMS Notification, Push Notification) from the base class and override the send() method to reflect appropriate behavior.
3. Use runtime polymorphism to call the send() method using a base class reference or interface, so that the actual message sending behavior is determined at runtime.
4. Demonstrate how this design allows easy addition of new notification types (e.g., Slack Notification) without changing the existing codebase logic.

Base class

```
class Notification:
```

```
    def send(self, message):
```

```
        raise NotImplementedError("Subclasses must override this method")
```

Email Notification

```
class EmailNotification(Notification):
```

```
    def send(self, message):
```

```
        print(f"Sending Email: {message}")
```

SMS Notification

```
class SMSNotification(Notification):
```

```
    def send(self, message):
```

```
        print(f"Sending SMS: {message}")
```

Push Notification

```
class PushNotification(Notification):
```

```
    def send(self, message):
```

```
        print(f"Sending Push Notification: {message}")
```

Adding a new type: Slack Notification

```
class SlackNotification(Notification):
```

```
    def send(self, message):
```

```

        print(f"Sending Slack Message: {message}")
# Function to send notifications - Demonstrates runtime polymorphism
def notify_user(notification_type, message):
    notification_type.send(message)
# Main execution
if __name__ == "__main__":
    # Create different notification types
    email = EmailNotification()
    sms = SMSNotification()
    push = PushNotification()
    slack = SlackNotification()
    # Polymorphic usage
    notify_user(email, "Welcome to the platform!")
    notify_user(sms, "Your OTP is 123456")
    notify_user(push, "You have a new friend request.")
    notify_user(slack, "Project updates are posted in #general")

```

Practice Assignment-1:

```

# Base Class - Shape
class Shape:
    def __init__(self, shape_id, color):
        self._shape_id = shape_id    # Encapsulated (protected)
        self._color = color          # Encapsulated (protected)
    def draw(self):
        raise NotImplementedError("Subclasses must implement draw() method")
    def display_details(self):

```

```

        raise NotImplementedError("Subclasses must implement display_details()
method")
# Derived Class - Circle
class Circle(Shape):
    def __init__(self, shape_id, color, radius):
        super().__init__(shape_id, color)
        self.__radius = radius        # Encapsulated (private)
    def draw(self):
        print(f"Drawing Circle - ID: {self._shape_id}, Color: {self._color}")
    def display_details(self):
        print(f"Circle Details -> ID: {self._shape_id}, Color: {self._color},
Radius: {self.__radius}")
# Derived Class - Rectangle
class Rectangle(Shape):
    def __init__(self, shape_id, color, width, height):
        super().__init__(shape_id, color)
        self.__width = width          # Encapsulated (private)
        self.__height = height        # Encapsulated (private)
    def draw(self):
        print(f"Drawing Rectangle - ID: {self._shape_id}, Color: {self._color}")
    def display_details(self):
        print(f"Rectangle Details -> ID: {self._shape_id}, Color: {self._color},
Width: {self.__width}, Height: {self.__height}")
# Main Functionality: Demonstrating Polymorphism
if __name__ == "__main__":
    # Create shape objects
    circle1 = Circle(101, "Red", 5.5)
    rectangle1 = Rectangle(102, "Blue", 4.0, 3.0)

```

```

circle2 = Circle(103, "Green", 7.2)
# Store in a list of base class references (polymorphism)
shapes = [circle1, rectangle1, circle2]
# Loop through and invoke draw and display_details polymorphically
for shape in shapes:
    shape.draw()
    shape.display_details()
print("-" * 40)

```

Day-4

Beginner:

Exercise-1:

You are tasked with building a basic file handling feature for a text-processing application . Your objective is to create a program that does the following:

- Writes a predefined message — "Welcome to file handling!" — to a file named example.txt.
- Reads the content of the same file after writing and displays it on the console to confirm the write operation was successful.

File Handling Example

Step 1: Write to the file

with open("example.txt", "w") as file:

file.write("Welcome to file handling!")

print("Message written to 'example.txt' successfully.\n")

```
# Step 2: Read from the file
with open("example.txt", "r") as file:
    content = file.read()

# Step 3: Display the content
print("Content read from 'example.txt':")
print(content)
```

Exercise-2:

Write a program that reads two numbers from a file and divides them.
Handle both:

- File reading errors (e.g., file not found)
- Division errors (e.g., divide by zero)

```
def read_and_divide(filename):
    try:
        # Try opening the file
        with open(filename, 'r') as file:
            lines = file.readlines()

            # Check if file has at least two lines
            if len(lines) < 2:
                print("Error: File does not contain two numbers.")
                return

            # Try converting to numbers
            num1 = float(lines[0].strip())
            num2 = float(lines[1].strip())

            # Try performing division
            result = num1 / num2
```

```

        print(f"Result of division: {num1} / {num2} = {result}")
except FileNotFoundError:
    print(f"Error: File '{filename}' not found.")
except ValueError:
    print("Error: The file must contain valid numbers.")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
except Exception as e:
    print(f"Unexpected error: {e}")
# Run the function
read_and_divide("numbers.txt")

```

Intermediate:

Exercise-1:

You are developing a banking application. One of the business rules states that a customer must maintain a minimum balance of 1000 units in their account at all times. If a withdrawal would cause the balance to drop below 1000, the system should prevent the withdrawal and raise a custom exception. Write a program that:

- Defines a custom exception class called LowBalanceError.
- Contains a function withdraw (balance, amount) that:
 - Deducts the amount from the balance if it doesn't violate the rule.
 - Raises LowBalanceError if the resulting balance would be less than 1000.
- Accepts input or hardcodes values to simulate a withdrawal.
- Handles the custom exception and prints an appropriate error message.

```

# Custom exception class
class LowBalanceError(Exception):
    def __init__(self, message="Withdrawal denied: Balance cannot go below
1000 units."):
        self.message = message
        super().__init__(self.message)

# Function to perform withdrawal
def withdraw(balance, amount):
    if balance - amount < 1000:
        raise LowBalanceError
    else:
        balance -= amount
        print(f"Withdrawal successful! New balance: {balance}")
    return balance

# Main code to simulate the operation
def main():
    # Hardcoded values (you can replace with input() for dynamic interaction)
    current_balance = 5000
    withdrawal_amount = 4500
    print(f"Current Balance: {current_balance}")
    print(f"Attempting to withdraw: {withdrawal_amount}")
    try:
        new_balance = withdraw(current_balance, withdrawal_amount)
    except LowBalanceError as e:
        print(f"Error: {e}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

```

```
# Run the program
if __name__ == "__main__":
    main()
```

Exercise-2:

You are building a tool to process multiple text files in a batch. Each file is expected to contain some data on its first line (such as a heading, title, or configuration). However, not all files may exist, and some may be empty due to previous processing failures or user errors. Your task is to:

- Accept a list of file names.
- For each file in the list:
 - If the file does not exist, print: " not found"
 - If the file exists but is empty, raise a custom exception named Empty File Error and catch it.
 - If the file contains data, read and print the first line only.

```
import os

# Custom Exception
class EmptyFileError(Exception):
    def __init__(self, filename):
        self.filename = filename
        super().__init__(f'{filename} is empty.')

# Function to process files
def process_files(file_list):
    for file in file_list:
        if not os.path.exists(file):
            print(f'{file}: not found')
        else:
```

```

try:
    with open(file, 'r') as f:
        content = f.readline().strip()
        if not content:
            raise EmptyFileError(file)
        print(f'{file}: {content}')
except EmptyFileError as e:
    print(f'{file}: {e}')

# Example list of filenames
files = ["file1.txt", "file2.txt", "file3.txt"]

# Process the files
process_files(files)

```

Advanced:

Exercise-1:

You are developing a reliable logging system for a backend application. Logs are essential for debugging and monitoring, so failure to write logs must be gracefully handled. Design a system that:

1. Attempts to write each log message to a primary file (app.log).
2. If writing fails due to an IOError (e.g., file is locked, unavailable, or disk is full), the system:
 - Retries writing up to 3 times.
 - Waits briefly between retries (optional: sleep for 0.5–1 second).
3. If all retry attempts fail, it writes the log message to a fallback file, named backup.log.

```
import time
```

```
def write_log(message, primary_file='app.log', fallback_file='backup.log',
retries=3, delay=1):
```

```

attempt = 0
while attempt < retries:
    try:
        with open(primary_file, 'a') as file:
            file.write(message + '\n')
        print(f"Log written to {primary_file}")
        return
    except IOError as e:
        print(f"Attempt {attempt + 1}: Failed to write to {primary_file} - {e}")
        time.sleep(delay)
        attempt += 1
# If all retries failed, write to fallback
try:
    with open(fallback_file, 'a') as fallback:
        fallback.write(message + '\n')
    print(f"Log written to fallback: {fallback_file}")
except IOError as e:
    print(f"Critical: Failed to write to both primary and fallback logs - {e}")
# Example usage
log_messages = [
    "System started",
    "Database connected",
    "Error: Unable to load configuration",
    "Warning: Low memory",
    "System shutting down"
]
for msg in log_messages:

```

```
write_log(msg)
```

Practice Assignment-1:

```
import os
```

```
# Book class
```

```
class Book:
```

```
    def __init__(self, book_id, title, author):
```

```
        self.book_id = book_id
```

```
        self.title = title
```

```
        self.author = author
```

```
    def __str__(self):
```

```
        return f"ID: {self.book_id}, Title: '{self.title}', Author: {self.author}"
```

```
    def to_line(self):
```

```
        return f"{self.book_id},{self.title},{self.author}\n"
```

```
    @staticmethod
```

```
    def from_line(line):
```

```
        try:
```

```
            book_id, title, author = line.strip().split(",", 2)
```

```
            return Book(int(book_id), title, author)
```

```
        except ValueError:
```

```
            raise ValueError("Invalid book data format")
```

```
# Library class
```

```
class Library:
```

```
    def __init__(self):
```

```
        self.books = []
```

```
    def add_book(self, book):
```

```

        self.books.append(book)
def save_to_file(self, filename):
    try:
        with open(filename, 'w') as file:
            for book in self.books:
                file.write(book.to_line())
            print(f"Saved {len(self.books)} book(s) to '{filename}'")
    except IOError as e:
        print(f" Error saving books to file: {e}")
def load_from_file(self, filename):
    if not os.path.exists(filename):
        print(f"File '{filename}' does not exist.")
        return
    try:
        with open(filename, 'r') as file:
            self.books = []
            for line in file:
                if line.strip():
                    try:
                        book = Book.from_line(line)
                        self.books.append(book)
                    except ValueError as ve:
                        print(f"Skipping invalid line: {line.strip()} ({ve})")
            print(f"Loaded {len(self.books)} book(s) from '{filename}'")
    except IOError as e:
        print(f"Error reading file: {e}")

```

```

def display_books(self):
    if not self.books:
        print("No books available.")
    else:
        print("Book List:")
        for book in self.books:
            print(f" - {book}")

# Sample usage
if __name__ == "__main__":
    library = Library()
    while True:
        print("\n--- Library Menu ---")
        print("1. Add Book")
        print("2. Save Books to File")
        print("3. Load Books from File")
        print("4. Display Books")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
        if choice == "1":
            try:
                book_id = int(input("Enter Book ID: "))
                title = input("Enter Title: ")
                author = input("Enter Author: ")
                library.add_book(Book(book_id, title, author))
                print("Book added.")
            except ValueError:
                print("Invalid input. Book ID must be an integer.")

```

```

elif choice == "2":
    library.save_to_file("books.txt")
elif choice == "3":
    library.load_from_file("books.txt")
elif choice == "4":
    library.display_books()
elif choice == "5":
    print("Exiting Library System.")
    break
else:
    print("Invalid choice. Please enter a number between 1 and 5.")

```

Day-5

Beginner:

Exercise-1:

Design a reusable function that can calculate the average of two numbers (int, float, or double)

Implement a template or generic function that takes two numbers and returns their average. If feasible, compare how templates/generics are handled differently in C++, Java, and Python.

```
from typing import Union
```

```
def calculate_average(a: Union[int, float], b: Union[int, float]) -> float:
```

```
    Returns the average of two numbers (int, float).
```

```
    try:
```

```
        return (a + b) / 2
```

```
    except TypeError:
```

```

        print("Error: Invalid input types. Both inputs must be numbers.")
    return None

# Example Usage
print(calculate_average(10, 20))
print(calculate_average(3.5, 6.5))
print(calculate_average(10, "twenty"))

```

Exercise-2:

You are building a notification system that must support different types of alert content, such as text messages(string like "Disk space low"), numeric codes (like Integer such as 404 etc.), or status flags(e.g., true to indicate active alert, false for no alert).

To ensure scalability, type safety, and code reusability, design a generic notification handler that can send alerts of any data type using a single class or function.

Your solution should:

- Support multiple data types without duplicating logic
- Include a method to send or display the notification
- Demonstrate usage with at least two different types (e.g., a message and a code).

```

from typing import TypeVar, Generic

# Define a generic type variable
T = TypeVar('T')

class NotificationHandler(Generic[T]):
    def __init__(self, alert_content: T):
        self.alert_content = alert_content

    def send_notification(self):
        print(f"Notification: {self.alert_content}")

```

Intermediate:

Exercise-1:

You are working on a software module for an IoT data aggregator system that collects data from various sensors deployed in a smart environment (like smart homes or smart factories). These sensors send data in different numeric formats, such as:

- Temperature sensors sending float values (e.g., 22.5°C)
- Pressure sensors sending double values (e.g., 1013.25 hPa)
- Motion sensors sending int values (e.g., 0 or 1 to indicate presence or absence)

To simplify processing and reduce code duplication, you want to design a single, reusable utility function that can handle any numeric data type and print a standardized log message.

Write a function template named `process_data` that can accept a parameter of any numeric type (int, float, or double) and print it in the format: `Sensor reading: <value>`

```
from typing import Union

# Define the function with a generic numeric type
def process_data(value: Union[int, float]) -> None:
    print(f'Sensor reading: {value}')

# Example usage
if __name__ == "__main__":
    motion_sensor = 1          # int
    temperature_sensor = 22.5  # float
    pressure_sensor = 1013.25  # float (Python has no double; float == C
                                double)

    process_data(motion_sensor)
    process_data(temperature_sensor)
```

```
process_data(pressure_sensor)
```

Exercise-2:

In a cloud-based system, only numeric data types should be allowed for upload (e.g., sensor values like Integer, Double). You want to restrict uploads to these types.

Write a bounded generic method `upload(T data)` in Java that prints “Uploading:”. Show usage with Integer and Double. Try using String and explain what happens.

```
public class CloudUploader {  
    // Bounded generic method to accept only subclasses of Number  
    public static <T extends Number> void upload(T data) {  
        System.out.println("Uploading: " + data);  
    }  
    public static void main(String[] args) {  
        // Valid uploads  
        upload(42);      // Integer  
        upload(98.6);    // Double  
        // Invalid upload (Uncommenting the below line causes compile-time  
        // error)  
        // upload("Hello"); // String is NOT a subclass of Number  
    }  
}
```

Advanced:

Exercise-1:

Build a modular, generic component framework for an embedded system. This system is deployed across different environments where

the behavior of certain components (like logging, memory allocation, encryption, etc.) needs to be customized at compile-time — without using virtual functions or runtime polymorphism, to ensure zero runtime overhead and maximum performance. You decide to use the policy-based design pattern — a powerful language (like C++)idiom that allows behavior to be injected into classes using template parameters, achieving static polymorphism (i.e., behavior determined at compile time).

Create a generic class template `Component<policy>` that delegates specific operations (like logging) to a policy class. Based on the policy passed during template instantiation, the component should behave differently.

```
#include <iostream>
```

```
// --- Logging Policies ---
```

```
class ConsoleLogger {
```

```
public:
```

```
    void log(const std::string& message) const {
```

```
        std::cout << "[Console] " << message << std::endl;
```

```
    }
```

```
};
```

```
class NoLogger {
```

```
public:
```

```
    void log(const std::string&) const {
```

```
        // No-op: Logging is disabled
```

```
    }
```

```
};
```

```

class DebugLogger {
public:
    void log(const std::string& message) const {
        std::cout << "[Debug] >>> " << message << std::endl;
    }
};

```

```

// --- Generic Component Template ---
template <typename LoggingPolicy>
class Component {
private:
    LoggingPolicy logger;

public:
    void runOperation() {
        logger.log("Operation started.");
        // Simulate some operation
        logger.log("Operation in progress...");
        logger.log("Operation finished.");
    }
};

```

Practice Assignment-1:

```

from typing import TypeVar, Generic, List
# Define a type variable for generic type
T = TypeVar('T')

```

```

class Queue(Generic[T]):
    def __init__(self):
        self.items: List[T] = []
    def enqueue(self, item: T) -> None:
        """Add an element to the rear of the queue."""
        self.items.append(item)
        print(f"Enqueued: {item}")
    def dequeue(self) -> T:
        """Remove and return the front element."""
        if self.is_empty():
            raise IndexError("Queue is empty. Cannot dequeue.")
        item = self.items.pop(0)
        print(f"Dequeued: {item}")
        return item
    def front(self) -> T:
        """Return the front element without removing it."""
        if self.is_empty():
            raise IndexError("Queue is empty. Cannot view front.")
        print(f"Front element: {self.items[0]}")
        return self.items[0]
    def is_empty(self) -> bool:
        """Check if the queue is empty."""
        return len(self.items) == 0

# Main function to test queue
def main():
    print("Queue with Integers:")
    int_queue = Queue[int]()

```

```
int_queue.enqueue(10)
int_queue.enqueue(20)
int_queue.front()
int_queue.dequeue()
int_queue.front()
print(f"Is queue empty? {int_queue.is_empty()}")
int_queue.dequeue()
print(f"Is queue empty? {int_queue.is_empty()}")
print("\nQueue with Strings:")
str_queue = Queue[str]()
str_queue.enqueue("hello")
str_queue.enqueue("world")
str_queue.front()
str_queue.dequeue()
str_queue.front()
print("\nQueue with Floats:")
float_queue = Queue[float]()
float_queue.enqueue(3.14)
float_queue.enqueue(2.71)
float_queue.front()
if __name__ == "__main__":
    main()
```